DEPLOYMENT PLAN

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Executive Summary

SettleOps AI is delivered as a fully self-hosted, on-premise software product for motor insurance carriers. The platform — a multi-agent decision-support system that reduces a 2-4 day manual claim review to a 10-minute AI-assisted decision — is shipped as a containerised stack that runs entirely inside the customer's data centre or private cloud (VPC).

No claim data, no policy data, no document leaves the customer's network perimeter.

Attribute	Value
Deployment model	Self-hosted, on-premise or customer-owned VPC
Tenancy	Single-tenant per customer install
Data residency	100% inside customer infrastructure
Internet egress	Optional (Tier 2) or fully air-gapped (Tier 1)
Target environment	Linux + Docker / Kubernetes
Pilot duration	8 weeks
Full production rollout	16-20 weeks from contract signing

2. Why Self-Hosted

Motor insurance is a regulated industry handling PII (NRIC, addresses, phone numbers), financial data (policy values, payouts), and legally privileged documents (police reports, adjuster reports). SettleOps AI is architected for self-hosting because:

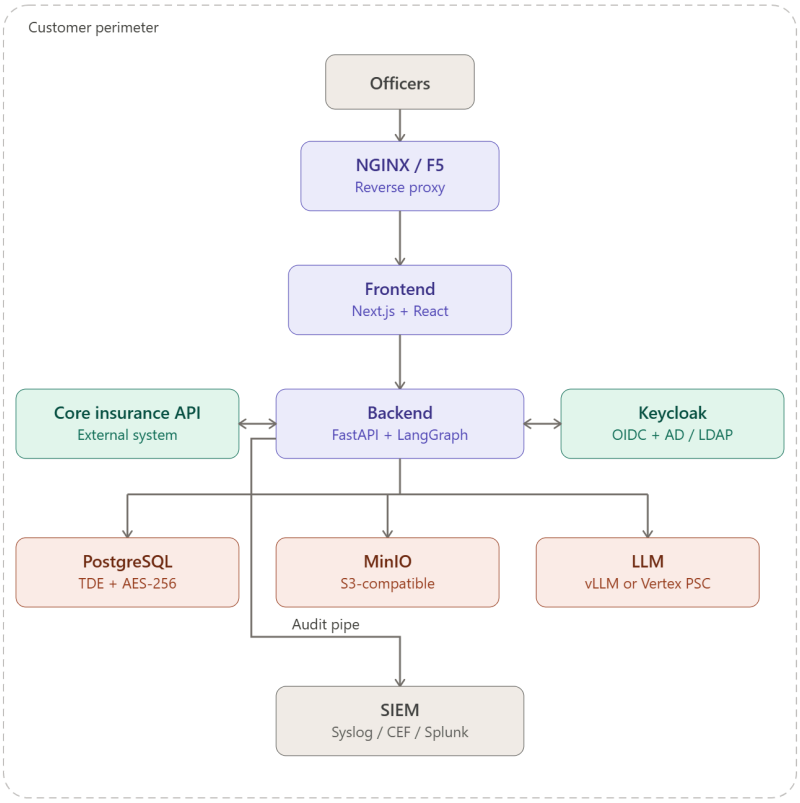
1. **Regulatory alignment:** Bank Negara Malaysia *Risk Management in Technology (RMiT)* policy and the *Personal Data Protection Act 2010 (PDPA)* require data residency, controlled access, and

© UMHackathon 2026

full audit trails.

2. **Customer control**: The insurer owns the data plane, the AI prompts, the audit logs, and the keys.
3. **No vendor lock-in on inference**: Deployable with either a customer-controlled cloud LLM endpoint or a locally-hosted open-weights model (see §5).
4. **Trust**: Claim data never traverses third-party SaaS infrastructure.

3. References Architecture



4. What We Ship & How We Build It

The customer receives a single signed bundle — settleops-1.0.0-onprem.tgz, containing every artefact below. No public Internet download at install time.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Artifact	Purpose
settleops-frontend:1.0.0	OCI image — Next.js production build
settleops-backend:1.0.0	OCI image — FastAPI + agents
settleops-inference:1.0.0 *(Tier 1)*	OCI image — vLLM serving
llm-weights/*.safetensors *(Tier 1)*	Llama 3.1 70B / Qwen 2.5 72B
settleops-helm-chart-1.0.0.tgz	Kubernetes manifest
agent-prompts-1.0.0.tar.gz	Versioned, tunable agent prompts
db-migrations-*.sql	Alembic schema
sbom-*.cdx.json	CycloneDX 1.5 Software Bill of Materials

Build toolchain: Docker BuildKit (multi-stage, distroless) · pip-compile --generate-hashes for hash-pinned Python deps · Trivy + Grype CVE scan (zero CRITICAL/HIGH gate) · **Sigstore Cosign** image signing · **Syft** SBOM · **SLSA Level 3** provenance · GPG-signed bundle from offline HSM-rooted key.

Customer verification at install time:

```
gpg --verify settleops-1.0.0-onprem.tgz.sig

cosign verify --key release-pubkey.pem registry.insurer.local/settleops-backend:1.0.0

cosign verify-attestation --type slsaprovenance --key release-pubkey.pem ...
```

5. Infrastructure

Role	vCPU	RAM	Disk	GPU	Count
Backend + agents	16	64 GB	200 GB	—	2 (HA)
Frontend	4	16 GB	50 GB	—	2 (HA)
Postgres primary + replica	8	32 GB	1 TB	—	2
MinIO (erasure-coded)	4	16 GB	4 TB	—	3
Local LLM *(Tier 1)*	16	128 GB	500 GB	A100 8…	1-2
Reverse proxy + ops	8	32 GB	500 GB	—	2

OS: Ubuntu 22.04 / RHEL 9 (CIS-hardened) · Container runtime: Docker 24+ · Orchestration: Kubernetes 1.29+ or Docker Compose · TLS via customer PKI.

6. Deployment Tiers

Tier	Inference	Egress	Best For
1 — Air-Gapped	Local Llama / Qwen on customer GPU	None	Strict / RMiT / banking-grade

© UMHackathon 2026

Email: umhackathon@um.edu.my

2 — Private Cloud LLM	Gemini via Vertex AI Private Service Connect	Allowlist-only to Vertex	Customers with GCP relationship
3 — Hybrid	Local for sensitive clusters, Vertex for vision	Partial	Cost / quality balance

Selected via the LLM_PROVIDER=local|vertex|hybrid config flag — same codebase, no rebuild.

6.1 Current Demo Deployment (Hackathon)

For this hackathon/demo environment, we currently deploy:

Layer	Platform
Frontend	Vercel
Backend	Render

This setup is for demonstration and rapid iteration. The production target for insurer rollouts remains self-hosted/on-premise as described in this plan.

7. Security Architecture

Surface	Control
Data at rest	AES-256 (LUKS + Postgres TDE + MinIO SSE)
Data in transit	TLS 1.3, mTLS service-to-service
Secrets	HashiCorp Vault / customer KMS (no plaintext `.env`)

© UMHackathon 2026

Email: umhackathon@um.edu.my

PII fields *(NRIC, names, addresses)*	Column-level encryption + row-level RBAC
Identity	OIDC via Keycloak federated to AD / LDAP / SSO; MFA mandatory; least-privilege RBAC
Audit log	Append-only, hash-chained, streamed to customer SIEM
Compliance	PDPA 2010 · BNM RMiT · ISO 27001 · SOC 2 readiness
Hardening	CIS Docker + K8s benchmarks · distroless · default-deny network policies

Quarterly third-party penetration test; CRITICAL CVEs patched within 72 h.

8. Integration with Insurer Systems

The most scrutinized area for a strict insurer. Twelve integration surfaces, grouped:

8.1 Integration Map

Integration	Direction	Protocol	Auth
Core platform (Guidewire / Sapiens / in-house)	Bi	REST + JSON / SOAP	mTLS + OAuth2 client credentials
Policy DB read replica	In	JDBC	Read-only user + mTLS
Document mgmt (SharePoint /	In	REST / CMIS	OAuth2 / Kerberos

© UMHackathon 2026

OpenText)			
Identity (AD / Azure AD / Okta)	In	OIDC / SAML 2.0	Standard OIDC
SIEM (Splunk / QRadar / Sentinel)	Out	Syslog / Splunk HEC / CEF	mTLS + token
Email gateway	Out	SMTP-TLS	SMTP auth
E-signature	Bi	REST + webhook	OAuth2
Adjuster mobile app	In	REST + SSE	mTLS + device cert
Workshop quote portal	In	REST	API key + IP allowlist
Payout / banking	Out	REST / ISO 20022	mTLS + customer-signed JWT
Message bus (optional)	Bi	Kafka / RabbitMQ	mTLS + ACL

8.2 Policy Data Access

Pattern	When	Notes
A — Live REST/SOAP API	Insurer exposes a policy lookup endpoint	mTLS to customer API gateway · 5-min encrypted in-memory TTL · pause-don't-guess on outage
B — Read-only DB replica	No API, or API too slow	`settleops_ro` user · field-level allowlist · per-claim audit row

© UMHackathon 2026

C — Nightly CDC mirror	Strict latency / mainframe	Debezium → encrypted local Postgres · 60-min lag triggers escalation
D — SFTP file drop	Legacy AS/400 / COBOL	GPG-encrypted CSV · hash totals reconciled

Cross-cutting: least-privilege DB user · field allowlist (no `dob`, no `customer_email` beyond what's needed) · every read appended to the hash-chained audit log · PDPA §38 erasure honored within 30 days.

8.3 Service-to-Pipeline Communication

- **Sync REST** for officer UI and live lookups — `Idempotency-Key` header on all state-changing calls.
- **SSE** for live agent telemetry — `Last-Event-ID` resume from LangGraph checkpoint.
- **Async message queue** for adjuster reports, workshop quotes, payouts — uses the **transactional outbox pattern** for exactly-once delivery semantics.

Suggested topics: `claim.received` · `claim.decision_drafted` · `claim.escalated` · `adjuster.report_requested` · `payout.authorized`.

8.4 API Contract

Versioned OpenAPI 3.1 spec at `/api/v1/openapi.json`. Core endpoints:

Endpoint	Purpose
`POST /api/v1/cases`	Create claim
`POST /api/v1/cases/{id}/documents`	Upload document
`POST /api/v1/cases/{id}/start`	Trigger PEVR pipeline

© UMHackathon 2026

`GET /api/v1/cases/{id}`	Retrieve case state
`GET /api/v1/cases/{id}/stream`	SSE telemetry
`POST /api/v1/cases/{id}/officer-action`	Approve / Decline / Challenge
`POST /api/v1/adjuster/{id}/report`	Adjuster upload
`POST /api/v1/workshop/{id}/quote`	Workshop quote

Outbound webhooks (`claim.decision_drafted`, `payout.authorize`, `audit.event`, …) are signed with a customer-supplied JWT key and retried with exponential backoff to a 24-hour dead-letter.

8.5 Policy Fit — How Customer Policies Become Agent Behavior

Three configurable layers, set up in Phase 2:

1. **Canonical Policy Schema** — normalized YAML (`product_code`, `cover.own_damage.{sum_insured,excess}`, `endorsements[*]`, …). All sources map into this.
2. **Source Adapter** — per-insurer YAML mapping native columns to canonical fields, e.g. `policy.cover.own_damage.excess: POLICY_COVER.OD_EXCESS_RM`.
3. **Policy Rule Pack** — versioned YAML rules the Policy Agent applies, reviewable by Compliance:

- id: R-001

description: "Flood damage requires 'flood' cover or FLOOD-EXT endorsement"

when: { incident.type: flood }

require_any:

- policy.cover.flood.covered == true
- policy.endorsements[*].code contains "FLOOD-EXT"

on_fail: decline

© UMHackathon 2026

citation: "Policy Schedule §4.2"

Every rule change triggers regression replay against the 200-case acceptance suite.

9. Install (Kubernetes)

```
tar -xzf settleops-1.0.0-onprem.tgz && cd settleops-1.0.0/
gpg --verify settleops-1.0.0.tgz.sig
./scripts/load-images.sh --registry registry.insurer.local
cp values.example.yaml values.yaml          # set TLS, DB, LLM tier, OIDC, SIEM
helm install settleops ./charts/settleops -f values.yaml -n settleops --create-namespace
./scripts/smoke-test.sh --base-url https://settleops.insurer.local
./scripts/run-uat.sh
```

Key config values: `LLM_PROVIDER` · `LLM_ENDPOINT` · `DATABASE_URL` · `OIDC_ISSUER_URL` · `SIEM_SYSLOG_HOST` · `OBJECT_STORE_ENDPOINT` · `CORE_INSURANCE_API_URL` · `ENCRYPTION_KEY_REF`.

10. HA / DR

Target	Value
Uptime SLO	99.5%
RPO (Recovery Point Objective)	≤ 15 min
RTO (Recovery Time Objective)	≤ 4 h
Postgres	Streaming replication + Patroni failover

© UMHackathon 2026

Email: umhackathon@um.edu.my

Object store	MinIO erasure coding (3+ nodes)
Backup	Hourly WAL · daily full · 30 d on-site / 90 d off-site · quarterly tabletop, annual full failover drill

11. Phased Rollout

Phase	Weeks	Activity	Exit Criteria
0 — Discovery	1-2	Workshops, infra survey, security questionnaire	Signed scope + ADR
1 — Build	3-4	Provision dev + UAT, install stack, integrate IdP + SIEM	Reachable from officer LAN
2 — Tuning	5-7	Map policy schema, tune prompts on 50 historical claims	≥ 90% replay accuracy
3 — UAT (shadow)	8-10	2-5 officers run AI in parallel, manual decision still official	Officer satisfaction ≥ 4/5
4 — Pilot	11-14	One branch / one product takes AI-assisted decisions for real	< 2% overturn rate
5 — Rollout	15-20	All branches; training; runbook handover	Full motor queue

© UMHackathon 2026

6 — Steady state	Ongoing	Self-improvement flywheel	Threshold raised over time

12. Risk Register (Top 5)

Risk	Mitigation
Local LLM accuracy below Gemini Flash	Tier 2 fallback + benchmark accuracy floor
Officer adoption resistance	Shadow-mode pilot + champions program
Legacy core platform integration	Adapter pattern + SFTP fallback
Compliance gap	ISO 27001 + RMiT pre-audit checklist
Air-gap blocks updates	Signed offline update bundles

13. Cost (Indicative)

Line	One-time	Annual
License (Tier 1, unlimited claims)	—	RM 600,000
Implementation services	RM 350,000	—
Hardware (GPU + servers)	RM 450,000	—

© UMHackathon 2026

Email: umhackathon@um.edu.my

Support (Sev-1 24×7)	—	RM 120,000
Prompt-tuning	—	RM 80,000
Year 1 total	**RM 800,000**	**RM 800,000**
Year 2+ total	—	**RM 800,000**

Break-even ~256 claims at RM 200 manual vs. RM 5 automated marginal cost.

14. Acceptance Criteria

1. ≥ 90% accuracy on the 200-case regression suite vs. historical decisions
2. End-to-end claim decision ≤ 90 s p95
3. Audit log integrity (hash chain + SIEM ingestion) verified
4. DR drill restores within RTO ≤ 4 h
5. Penetration test report — zero CRITICAL/HIGH findings
6. Officer satisfaction ≥ 4/5 in pilot

© UMHackathon 2026

Email: umhackathon@um.edu.my